

Week 2 – Python Basics

Python basics

This first lab session is an introduction to Python, so after the instructions on how to install Python, we will briefly show/review some Python basics with code samples, and see how to work with simple data types, such as numbers, strings and lists.

Python provides arithmetic operations for manipulating numbers in a straightforward manner that is similar to other programming languages. The following examples involve arithmetic operations on integers:

PLEASE NOTE: “>>>” is not a command, but indicates the line you are writing on (only present if you are using ipython).

Try the following and compare your result with the expected (ignore “>>>”, you must not type it)

```
>>> 2+2
```

```
4
```

```
>>> 4/3
```

```
1
```

```
>>> 3*8
```

```
24
```

The following example assigns numbers to two variables and computes their product:

```
>>> x = 4
```

```
>>> y = 7
```

```
>>> x * y
```

```
28
```

The following examples demonstrate arithmetic operations involving integers:

```
>>> 2+2
```

```
4
```

```
>>> 4/3
```

```
1
```

```
>>> 3*8
```

```
24
```

Notice that division ("/") of two integers is actually truncation in which only the integer result is retained. The following example converts a floating point number into exponential form:

```
>>> fnum = 0.00012345689000007
```

```
>>> "%.14e"%fnum
```

```
'1.23456890000070e-04'
```

You can use the `int()` function and the `float()` function to convert strings to numbers:

```
word1 = "123"
```

```
word2 = "456.78"
```

```
var1 = int(word1)
```

```
var2 = float(word2)
```

```
print("var1: ",var1," var2: ",var2)
```

The output from the preceding code block is here:

```
var1: 123    var2: 456.78
```

Alternatively, you can use the `eval()` function:

```
word1 = "123"
```

```
word2 = "456.78"
```

```
var1 = eval(word1)
```

```
var2 = eval(word2)
```

```
print("var1: ",var1," var2: ",var2)
```

A **string** in Python is a sequence of ASCII-encoded bytes. You can concatenate two strings using the "+" operator. The following example prints a string and then concatenates two single-letter strings:

```
>>> 'abc'
```

```
'abc'
```

```
>>> 'a' + 'b'
```

```
'ab'
```

You can use "+" or "*" to concatenate identical strings, as shown here:

```
>>> 'a' + 'a' + 'a'
```

```
'aaa'
```

```
>>> 'a' * 3
```

```
'aaa'
```

You can assign strings to variables and print them using the print command:

```
>>> print('abc')
```

```
abc
```

```
>>> x = 'abc'
```

```
>>> print(x)
```

```
abc
```

```
>>> y = 'def'
```

```
>>> print(x + y)
```

```
abcdef
```

You can “unpack” the letters of a string and assign them to variables, as shown here:

```
>>> str = "World"
```

```
>>> x1,x2,x3,x4,x5 = str
```

```
>>> x1
```

```
'W'
```

```
>>> x2
```

```
'o'
```

```
>>> x3
```

```
'r'
```

```
>>> x4
```

```
'l'
```

```
>>> x5
```

```
'd'
```

The preceding code snippets shows you how easy it is to extract the letters in a text string, and soon you will learn how to “unpack” other Python data structures. You can extract substrings of a string as shown in the following examples:

```
>>> x = "abcdef"
```

```
>>> x[0]
```

```
'a'
>>> x[-1]
'f'
>>> x[1:3]
'bc'
>>> x[0:2] + x[5:]
'abf'
```

However, you will cause an error if you attempt to “subtract” two strings, as you probably expect:

```
>>> 'a' - 'b'
```

This will display the following error (we will learn how to solve these errors in the next few weeks):

Traceback (most recent call last):

```
File "<stdin>", line 1, in <module>
```

```
TypeError: unsupported operand type(s) for -: 'str' and 'str'
```

Python Collections

In the previous sections, we have reviewed some basic concepts of programming in Python. In this part, we will briefly discuss a type of collections, i.e. lists. After you have finished reading this chapter, you will be in a better position to create more complex Python modules using one or more of these data structures. We will delve into the details in the next few weeks, so don't worry!

Lists

Python supports a **list** data type, along with a rich set of list-related functions. Since lists are not typed, you can create a list of different data types, as well as multidimensional lists. Now let's see how to manipulate list structures in Python.

A Python list consists of comma-separated values enclosed in a pair of square brackets. The following examples illustrate the syntax for defining a list in Python, and also how to perform various operations on a Python list:

```
>>> list = [1, 2, 3, 4, 5]
>>> list
[1, 2, 3, 4, 5]
>>> list[2]
```

3

```
>>> list2 = list + [1, 2, 3, 4, 5]
>>> list2
[1, 2, 3, 4, 5, 1, 2, 3, 4, 5]
>>> list2.append(6)
>>> list2
[1, 2, 3, 4, 5, 1, 2, 3, 4, 5, 6]
>>> len(list)
```

5

```
>>> x = ['a', 'b', 'c']
>>> y = [1, 2, 3]
>>> z = [x, y]
>>> z[0]
['a', 'b', 'c']
>>> len(x)
```

3

You can assign multiple variables to a list, provided that the number and type of the variables match the structure. Here is an example:

```
>>> point = [7,8]
>>> x,y = point
>>> x
```

7

```
>>> y
```

8

The following example shows you how to assign values to variables from a more complex data structure:

```
>>> line = ['a', 10, 20, (2020,10,31)]
>>> x1,x2,x3,date1 = line
>>> x1
'a'
```

```

>>> x2
10
>>> x3
20
>>> date1
(2020, 10, 31)

```

If you want to access the year/month/date components of the **date1** element in the preceding code block, you can do so with the following code block:

```

>>> line = ['a', 10, 20, (2020,10,31)]
>>> x1,x2,x3,(year,month,day) = line
>>> x1
'a'
>>> x2
10
>>> x3
20
>>> year
2020
>>> month
10
>>> day
31

```

If the number and/or structure of the variables do not match the data, an error message is displayed, as shown here:

```

>>> point = (1,2)
>>> x,y,z = point
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
ValueError: need more than 2 values to unpack

```

If the number of variables that you specify is less than the number of data items, you will see an error message, as shown here:

```
>>> line = ['a', 10, 20, (2014,01,31)]
>>> x1,x2 = line
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
ValueError: too many values to unpack
```